



Python Programming Fundamentals

For Loops

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. For Loop Syntax
2. The range() Function
3. Looping Over Lists and Strings
4. enumerate() — Index and Value Together
5. zip() — Loop Over Two Lists in Parallel
6. Nested Loops
7. List Comprehensions — A Sneak Peek



Outline

1. For Loop Syntax

2. The range() Function

3. Looping Over Lists and Strings

4. enumerate() — Index and Value Together

5. zip() — Loop Over Two Lists in Parallel

6. Nested Loops

7. List Comprehensions — A Sneak Peek



1. For Loop Syntax

A **for loop** iterates over every item in a sequence (list, string, range, ...).

```
for variable in sequence:  
    # body — executed once per item
```



1. For Loop Syntax

A **for loop** iterates over every item in a sequence (list, string, range, ...).

```
for variable in sequence:
```

```
    # body – executed once per item
```

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:
```

```
    print(fruit)
```

Output:

apple

banana

cherry



1. For Loop Syntax

A **for loop** iterates over every item in a sequence (list, string, range, ...).

```
for variable in sequence:
    # body — executed once per item

fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
cherry
```

The loop variable (`fruit`) takes on each value in turn. You choose the name — make it meaningful!



Outline

1. For Loop Syntax
- 2. The range() Function**
3. Looping Over Lists and Strings
4. enumerate() — Index and Value Together
5. zip() — Loop Over Two Lists in Parallel
6. Nested Loops
7. List Comprehensions — A Sneak Peek



2. The range() Function

`range()` generates a sequence of integers — perfect for counting loops.

```
range(stop)           # 0, 1, 2, ..., stop-1
range(start, stop)    # start, start+1, ..., stop-1
range(start, stop, step) # start, start+step, ...
```




2. The range() Function

`range()` generates a sequence of integers — perfect for counting loops.

```
range(stop)           # 0, 1, 2, ..., stop-1
range(start, stop)    # start, start+1, ..., stop-1
range(start, stop, step) # start, start+step, ...

for i in range(5):
    print(i)           # 0 1 2 3 4

for i in range(1, 6):
    print(i)           # 1 2 3 4 5

for i in range(0, 10, 2):
    print(i)           # 0 2 4 6 8

for i in range(10, 0, -1):
    print(i)           # 10 9 8 7 6 5 4 3 2 1
```



Outline

1. For Loop Syntax
2. The range() Function
- 3. Looping Over Lists and Strings**
4. enumerate() — Index and Value Together
5. zip() — Loop Over Two Lists in Parallel
6. Nested Loops
7. List Comprehensions — A Sneak Peek



3. Looping Over Lists and Strings

Looping over a list:

```
scores = [85, 92, 78, 95, 61]
```

```
total = 0
```

```
for score in scores:
```

```
    total += score
```

```
average = total / len(scores)
```

```
print(f"Average score: {average:.1f}")    # 82.2
```



3. Looping Over Lists and Strings



3. Looping Over Lists and Strings

Looping over a list:

```
scores = [85, 92, 78, 95, 61]
```

```
total = 0
```

```
for score in scores:  
    total += score
```

```
average = total / len(scores)
```

```
print(f"Average score: {average:.1f}")    # 82.2
```

Looping over a string – character by character:

```
word = "Python"
```

```
for char in word:  
    print(char, end=" ")    # P y t h o n
```



3. Looping Over Lists and Strings

```
vowels = 0
for char in word.lower():
    if char in "aeiou":
        vowels += 1
print(f"\n{word} has {vowels} vowels")    # 1 vowel
```



Outline

1. For Loop Syntax
2. The range() Function
3. Looping Over Lists and Strings
4. **enumerate() — Index and Value Together**
5. zip() — Loop Over Two Lists in Parallel
6. Nested Loops
7. List Comprehensions — A Sneak Peek



4. enumerate() — Index and Value Together

enumerate() gives you both the **index** and the **value** in each iteration.

```
fruits = ["apple", "banana", "cherry"]
```

```
for index, fruit in enumerate(fruits):  
    print(f"{index}: {fruit}")
```

Output:

```
0: apple  
1: banana  
2: cherry
```




4. enumerate() — Index and Value Together



4. enumerate() — Index and Value Together

enumerate() gives you both the **index** and the **value** in each iteration.

```
fruits = ["apple", "banana", "cherry"]
```

```
for index, fruit in enumerate(fruits):  
    print(f"{index}: {fruit}")
```

Output:

```
0: apple  
1: banana  
2: cherry
```

Start counting from 1:

```
for i, fruit in enumerate(fruits, start=1):  
    print(f"{i}. {fruit}")
```



4. enumerate() — Index and Value Together

```
# 1. apple  
# 2. banana  
# 3. cherry
```



Outline

1. For Loop Syntax
2. The range() Function
3. Looping Over Lists and Strings
4. enumerate() — Index and Value Together
- 5. zip() — Loop Over Two Lists in Parallel**
6. Nested Loops
7. List Comprehensions — A Sneak Peek



5. zip() — Loop Over Two Lists in Parallel

zip() pairs up elements from two (or more) sequences.

```
names = ["Alice", "Bob", "Carol"]  
scores = [88, 74, 95]
```

```
for name, score in zip(names, scores):  
    print(f"{name}: {score}")
```

Output:

```
Alice: 88  
Bob: 74  
Carol: 95
```



5. zip() — Loop Over Two Lists in Parallel



5. zip() – Loop Over Two Lists in Parallel

zip() pairs up elements from two (or more) sequences.

```
names = ["Alice", "Bob", "Carol"]  
scores = [88, 74, 95]
```

```
for name, score in zip(names, scores):  
    print(f"{name}: {score}")
```

Output:

Alice: 88

Bob: 74

Carol: 95

```
# Practical: calculate grades
```

```
grades = []
```

```
for name, score in zip(names, scores):
```



5. zip() — Loop Over Two Lists in Parallel

```
grade = "Pass" if score >= 60 else "Fail"  
grades.append((name, grade))
```

```
for name, grade in grades:  
    print(f"{name} — {grade}")
```




Outline

1. For Loop Syntax
2. The range() Function
3. Looping Over Lists and Strings
4. enumerate() — Index and Value Together
5. zip() — Loop Over Two Lists in Parallel
- 6. Nested Loops**
7. List Comprehensions — A Sneak Peek



6. Nested Loops

A loop **inside** another loop — the inner loop runs fully for each iteration of the outer loop.

```
for row in range(1, 4):
    for col in range(1, 4):
        print(f"{row}×{col}={row*col}", end=" ")
    print()      # new line after each row
```

Output:

```
1×1=1  1×2=2  1×3=3
2×1=2  2×2=4  2×3=6
3×1=3  3×2=6  3×3=9
```



6. Nested Loops



6. Nested Loops

A loop **inside** another loop – the inner loop runs fully for each iteration of the outer loop.

```
for row in range(1, 4):
    for col in range(1, 4):
        print(f"{row}×{col}={row*col}", end=" ")
    print()      # new line after each row
```

Output:

```
1×1=1  1×2=2  1×3=3
2×1=2  2×2=4  2×3=6
3×1=3  3×2=6  3×3=9
```

Pattern printing:

```
rows = 5
for i in range(1, rows + 1):
```



6. Nested Loops

```
print("*" * i)
# *
# **
# ***
# ****
# *****
```



Outline

1. For Loop Syntax
2. The range() Function
3. Looping Over Lists and Strings
4. enumerate() — Index and Value Together
5. zip() — Loop Over Two Lists in Parallel
6. Nested Loops
- 7. List Comprehensions — A Sneak Peek**



7. List Comprehensions – A Sneak Peek

A concise way to build a list using a for loop in a single line.

```
# Traditional for loop
```

```
squares = []
```

```
for x in range(1, 6):
```

```
    squares.append(x ** 2)
```

```
# List comprehension – same result
```

```
squares = [x ** 2 for x in range(1, 6)]
```

```
print(squares) # [1, 4, 9, 16, 25]
```



7. List Comprehensions — A Sneak Peek



7. List Comprehensions – A Sneak Peek

A concise way to build a list using a for loop in a single line.

```
# Traditional for loop
```

```
squares = []
```

```
for x in range(1, 6):  
    squares.append(x ** 2)
```

```
# List comprehension – same result
```

```
squares = [x ** 2 for x in range(1, 6)]
```

```
print(squares) # [1, 4, 9, 16, 25]
```

With a condition:

```
# Only even squares
```

```
even_squares = [x ** 2 for x in range(1, 11) if x % 2 == 0]
```

```
print(even_squares) # [4, 16, 36, 64, 100]
```



7. List Comprehensions — A Sneak Peek

```
# Uppercase all words
words = ["hello", "world", "python"]
upper = [w.upper() for w in words]
print(upper)      # ['HELLO', 'WORLD', 'PYTHON']
```



Thanks for Watching - Any questions?